Briefing para produção audiovisual e de conteúdo
Aula Completa
2026.1

Professor: Luan Tavares Lourenço
Disciplina: Desenvolvimento Web

Aula:
1☐ 2☐ 3☒ 4☐ 5☐ 6☐ 7☐ 8☐ 9☐ 10☐ 11☐ 12☐ 13☐ 14☐ 15☐ 16☐

Título da aula:

Node.js: Tipos de Referências e loops

Opção do TA: Vídeo ☐Texto ☒

1 - Introdução

Até aqui, a base foi entender onde o JavaScript roda no backend e como o Node.js nos permite escrever programas fora do navegador. Agora, o próximo passo é entender melhor como a linguagem organiza valores em memória e como isso afeta o comportamento do código no dia a dia.

Nesta aula, o foco será nos tipos de referência, que são object, array e function. Antes disso, porém, vamos começar pelo conceito mais importante: o que é um objeto em JavaScript, por que quase tudo na linguagem se relaciona com objetos, e como isso ajuda a explicar a diferença entre tipos primitivos e tipos de referência. Em seguida, vamos conectar esse tema com comparação, cópia, spread operator, estruturas de repetição, métodos nativos dos tipos e, ao final, um primeiro contato com try/catch.

2 - O que é um objeto em JavaScript

Em JavaScript, objeto é uma estrutura que agrupa dados e comportamento. Em termos práticos, objeto é tudo aquilo que pode guardar propriedades e, em muitos

casos, também métodos. Um método nada mais é do que uma função associada a um objeto.

Exemplo:

```
const aluno = {
  nome: "Ana",
  idade: 21,
  apresentar: () => {
    return "Olá, eu sou Ana";
  }
};
```

Nesse caso, nome e idade são propriedades, e apresentar é um método. Essa ideia é central porque ela ajuda a entender um dos princípios mais importantes da linguagem: em JavaScript, muitos recursos são organizados como objetos, inclusive arrays e funções.

3 - JSON

JSON significa JavaScript Object Notation. Ele é um formato de texto usado para representar dados estruturados, normalmente em pares de chave e valor, de forma parecida com objetos JavaScript.

Exemplo de JSON:

```
{
  "nome": "Ana",
  "idade": 21,
  "ativo": true
}
```

É importante explicar que JSON parece um objeto JavaScript, mas não é exatamente a mesma coisa. JSON é um formato textual, muito usado para trocar dados entre sistemas, APIs, servidores e aplicações.

Ele é importante porque:

- é um padrão muito usado na comunicação entre frontend e backend;

- é muito comum em APIs;

- é muito usado para armazenar e transportar dados;

- no Node.js, aparece com frequência ao ler arquivos, receber requisições e responder dados.

Também vale mostrar os principais métodos do objeto JSON.

3.1 - JSON.parse()

Converte um texto em formato JSON para um valor JavaScript.

```
const texto = '{"nome":"Ana","idade":21}';
const objeto = JSON.parse(texto);

// { nome: "Ana", idade: 21 }
console.log(objeto);
```

3.2 - JSON.stringify()

Converte um valor JavaScript para texto no formato JSON.

```
const usuario = {
  nome: "Ana",
  idade: 21
};

const texto = JSON.stringify(usuario);

// {"nome":"Ana","idade":21}
console.log(texto);
```

Didaticamente, esse é um ótimo ponto para reforçar:

- objeto JavaScript é estrutura da linguagem;

- JSON é formato de texto para representar dados;

- JSON.parse() transforma texto em valor JavaScript;

- JSON.stringify() transforma valor JavaScript em texto JSON.

4 - Tipos primitivos e tipos de referência

A partir da ideia de objeto, fica mais fácil entender a diferença entre tipo primitivo e tipo de referência.

Tipos primitivos representam valores mais simples, como:

- string
- number
- boolean
- null
- undefined
- bigint
- symbol

Já os tipos de referência são:

- object
- array
- function

A diferença prática é esta:

- Em tipos primitivos, a variável guarda diretamente o valor.

- Em tipos de referência, a variável guarda uma referência para um valor em memória.

Essa distinção é base para entender comparação, cópia e mutação de dados em JavaScript.

5 - Stack e heap: uma visão didática

Didaticamente, vale imaginar a memória em duas regiões conceituais:

- Stack: área mais “rápida” e organizada, onde podemos imaginar os identificadores locais e valores simples sendo manipulados.

- Heap: área onde vivem estruturas mais complexas, como objetos, arrays e funções.

Sem transformar isso numa explicação de arquitetura de baixo nível, a imagem didática útil para a aula é esta:

- a variável fica com uma “anotação” de onde está o dado;

- o objeto em si fica em outro lugar da memória;

- então duas variáveis podem apontar para o mesmo valor de referência.

Exemplo:

```
const a = { nome: "Maria" };
const b = a;

b.nome = "João";

// João
console.log(a.nome);

// João
console.log(b.nome);
```

Aqui, a e b apontam para a mesma referência. Não existem dois objetos independentes; existe um objeto só, com duas variáveis apontando para ele.

6 - Comparação de tipos de referência

Esse é o clássico problema que costuma confundir quem está começando:

```
const obj1 = { valor: 10 };
const obj2 = { valor: 10 };

// false
```

```
console.log(obj1 === obj2);
```

Mesmo tendo o mesmo conteúdo, o resultado é false. O motivo é que, em tipos de referência, a comparação verifica se as variáveis apontam para a mesma referência, e não se o conteúdo “parece igual”.

Por outro lado:

```
const obj1 = { valor: 10 };
const obj2 = obj1;

// true
console.log(obj1 === obj2);
```

Agora o resultado é true, porque os dois nomes apontam para o mesmo objeto em memória.

7 - Cópia de tipos de referência

Outro ponto importante: copiar uma variável de referência não cria automaticamente uma cópia independente do valor.

```
const original = { nome: "Carlos" };
const copia = original;

copia.nome = "Fernanda";

// Fernanda
console.log(original.nome);
```

Isso acontece porque copia não é uma nova estrutura; é apenas outra referência para o mesmo objeto.

8 - Copy

Quando falamos em copy, estamos falando do ato de criar uma cópia de um valor de referência para evitar que duas variáveis apontem para o mesmo dado.

Sem cópia:

```
const usuario1 = { nome: "Ana" };
const usuario2 = usuario1;

usuario2.nome = "Bruna";

// Bruna
console.log(usuario1.nome);

// Bruna
console.log(usuario2.nome);
```

Nesse caso, não houve uma cópia real. As duas variáveis continuam apontando para o mesmo objeto em memória.

Quando queremos uma cópia separada, precisamos criar uma nova estrutura. Uma das formas mais comuns de fazer isso em JavaScript é com o spread operator, que veremos a seguir.

9 - Spread operator

O spread operator (...) é um operador que espalha valores. Ele é muito útil para copiar e combinar arrays e objetos.

Exemplo com objeto:

```
const usuario = { nome: "Lia", idade: 30 };
const copiaUsuario = { ...usuario };

copiaUsuario.nome = "Bruna";

// Lia
console.log(usuario.nome);

// Bruna
console.log(copiaUsuario.nome);
```

Exemplo com array:

```
const numeros = [1, 2, 3];
const copiaNumeros = [...numeros];

copiaNumeros.push(4);

// [1, 2, 3]
console.log(numeros);

// [1, 2, 3, 4]
console.log(copiaNumeros);
```

Mas é importante destacar: o spread faz cópia superficial.

```
const pessoa = {
  nome: "Rafa",
  endereco: { cidade: "Recife" }
};

const copiaPessoa = { ...pessoa };
copiaPessoa.endereco.cidade = "Olinda";

// Olinda
console.log(pessoa.endereco.cidade);
```

O objeto externo foi copiado, mas o objeto interno continuou sendo compartilhado.

10 - Object: estrutura de chave e valor

O tipo object é a estrutura genérica de chave e valor da linguagem. Ele serve para modelar entidades, configurações, registros e agrupamentos de informação.

Exemplo:

```
const produto = {
  nome: "Notebook",
  preco: 3500,
```

```
  emEstoque: true
};
```

Acesso por ponto:

```
// Notebook
console.log(produto.nome);
```

Acesso por colchetes:

```
// 3500
console.log(produto["preco"]);
```

Adição e alteração de propriedades:

```
produto.marca = "Lenovo";
produto.preco = 3200;
```

Remoção:

delete produto.emEstoque;

Alguns métodos e recursos importantes para mostrar:

- Object.keys(obj) -> lista as chaves
- Object.values(obj) -> lista os valores
- Object.entries(obj) -> lista pares chave/valor
- hasOwnProperty() -> verifica se a propriedade existe no próprio objeto

Exemplo:

```
const aluno = {
  nome: "Bia",
  nota: 9
};

// ["nome", "nota"]
```

```
console.log(Object.keys(aluno));

// ["Bia", 9]
console.log(Object.values(aluno));

// [["nome", "Bia"], ["nota", 9]]
console.log(Object.entries(aluno));
```

11 - API e vendor API

Antes de seguir, aqui cabe apresentar a sigla API.

API significa Application Programming Interface. Em termos práticos, é uma interface pronta de recursos que podemos usar. Quando falamos de métodos e propriedades disponíveis em um tipo, estamos falando da API daquele tipo.

Exemplo:

- String tem sua API
- Number tem sua API
- Array tem sua API
- Object tem sua API

Quando você usa algo como toUpperCase(), toFixed(), map() ou filter(), você está usando a API disponibilizada por aquele tipo.

Aqui você também pode citar a ideia de vendor API. Didaticamente, vale explicar assim: além da linguagem JavaScript em si ter seus objetos e métodos nativos, o ambiente onde o JavaScript roda também pode oferecer APIs próprias. Ou seja:

- existe a API da própria linguagem;
- e existem APIs fornecidas pelo ambiente ou fornecedor daquele ambiente.

No navegador, por exemplo, existem APIs do navegador. No Node.js, existem APIs do próprio runtime. Então, quando falamos que “em JavaScript tudo se relaciona com objeto e com API”, a ideia é mostrar que os tipos possuem métodos próprios, e que o ambiente também disponibiliza recursos adicionais.

12 - Array: lista indexada e objeto especial

Array é um tipo de referência voltado para coleções ordenadas por índice. Em JavaScript, array não é tipo primitivo. Ele é uma estrutura própria para trabalhar com listas de valores.

Exemplo:

```
const frutas = ["maçã", "banana", "uva"];
```

Acesso por índice:

```
// maçã
console.log(frutas[0]);
```

Tamanho:

```
// 3
console.log(frutas.length);
```

Alteração:

```
frutas[1] = "laranja";
```

Métodos básicos:

- push() -> adiciona no final
- pop() -> remove do final
- unshift() -> adiciona no início
- shift() -> remove do início
- includes() -> verifica existência
- indexOf() -> retorna posição

Exemplo:

```
const numeros = [10, 20, 30];

numeros.push(40);
```

```
numeros.pop();

// true
console.log(numeros.includes(20));

// 2
console.log(numeros.indexOf(30));
```

13 - Métodos nativos e a ideia de “tudo é objeto”

Didaticamente, vale mostrar que em JavaScript muitos valores conseguem acessar propriedades e métodos porque a linguagem possui APIs prontas para cada tipo.

Exemplo com string:

```
const texto = "desenvolvimento web";

// 19
console.log(texto.length);

// DESENVOLVIMENTO WEB
console.log(texto.toUpperCase());

// true
console.log(texto.includes("web"));

// desenvolvimento
console.log(texto.slice(0, 14));
```

Exemplo com number:

```
const valor = 123.456;

// 123.46
console.log(valor.toFixed(2));

// 123.456
console.log(valor.toString());
```

```
// false
console.log(Number.isInteger(valor));
```

Exemplo com array:

```
const nomes = ["Ana", "Bruno", "Carla"];

// 3
console.log(nomes.length);

// Ana - Bruno - Carla
console.log(nomes.join(" - "));

// true
console.log(nomes.includes("Ana"));
```

A ideia importante aqui é: a linguagem oferece APIs internas para strings, numbers, arrays e objetos. No uso prático, isso aparece como métodos e propriedades que já vêm da própria linguagem e também do ambiente.

14 - Desestruturação de object e array

Antes de entrar em loops, vale apresentar a desestruturação, porque ela é muito usada no JavaScript moderno para extrair valores de objetos e arrays de forma mais prática.

14.1 - Desestruturação de object

Na desestruturação de objeto, retiramos propriedades e colocamos seus valores em variáveis.

Exemplo:

```
const usuario = {
  nome: "Ana",
  idade: 21
};
```

```
const { nome, idade } = usuario;

// Ana
console.log(nome);

// 21
console.log(idade);
```

Também é possível renomear as variáveis:

```
const produto = {
  nome: "Notebook",
  preco: 3500
};

const { nome: nomeProduto, preco: precoProduto } = produto;

// Notebook
console.log(nomeProduto);

// 3500
console.log(precoProduto);
```

14.2 - Desestruturação de array

Na desestruturação de array, retiramos valores com base na posição de cada elemento.

Exemplo:

```
const cores = ["azul", "verde", "preto"];

const [cor1, cor2, cor3] = cores;

// azul
console.log(cor1);
```

```
// verde
console.log(cor2);

// preto
console.log(cor3);
```

Também podemos ignorar posições:

```
const numeros = [10, 20, 30];

const [primeiro, , terceiro] = numeros;

// 10
console.log(primeiro);

// 30
console.log(terceiro);
```

Didaticamente, a desestruturação ajuda bastante porque deixa o código mais direto e mais legível, especialmente quando estamos lidando com objetos retornados de funções, arrays, parâmetros e dados vindos de APIs.

15 - Loops de repetição

Estruturas de repetição existem para executar um bloco de código várias vezes. Nesta aula, vale apresentar tanto os loops clássicos quanto os métodos de array que funcionam como forma moderna de percorrer coleções.

15.1 - for

```
for (let i = 0; i < 5; i++) {
  // 0
  // 1
  // 2
  // 3
  // 4
  console.log(i);
}
```

Boa opção quando precisamos de controle explícito do índice.

15.2 - while

```
let contador = 0;

while (contador < 3) {
  // 0
  // 1
  // 2
  console.log(contador);
  contador++;
}
```

Útil quando a repetição depende de uma condição mais aberta.

15.3 - for...of

```
const cores = ["azul", "verde", "preto"];

for (const cor of cores) {
  // azul
  // verde
  // preto
  console.log(cor);
}
```

Ótimo para iterar valores de arrays.

15.4 - for...in

```
const pessoa = {
  nome: "Lucas",
  idade: 25
};

for (const chave in pessoa) {
```

```
  // nome Lucas
  // idade 25
  console.log(chave, pessoa[chave]);
}
```

Mais apropriado para percorrer chaves de objetos.

16 - Principais métodos de array ligados a repetição

Aqui está uma das partes centrais da aula. Arrays têm métodos muito importantes para percorrer, transformar, filtrar e agregar dados.

16.1 - forEach()

Executa uma função para cada elemento.

```
const numeros = [1, 2, 3];

numeros.forEach((numero) => {
  // 1
  // 2
  // 3
  console.log(numero);
});
```

16.2 - map()

Cria um novo array transformando cada item.

```
const numeros = [1, 2, 3];
const dobrados = numeros.map((numero) => {
  return numero * 2;
});

// [2, 4, 6]
console.log(dobrados);
```

16.3 - filter()

Cria um novo array com os elementos que passam em um teste.

```
const numeros = [1, 2, 3, 4, 5];
const pares = numeros.filter((numero) => {
  return numero % 2 === 0;
});

// [2, 4]
console.log(pares);
```

16.4 - find()

Retorna o primeiro elemento que satisfaz a condição.

```
const usuarios = [
  { nome: "Ana", ativo: false },
  { nome: "Beto", ativo: true }
];

const encontrado = usuarios.find((usuario) => {
  return usuario.ativo;
});

// { nome: "Beto", ativo: true }
console.log(encontrado);
```

16.5 - some()

Verifica se ao menos um item atende à condição.

```
const notas = [4, 6, 8];

const existeAprovado = notas.some((nota) => {
  return nota >= 7;
});
```

```
// true
console.log(existeAprovado);
```

16.6 - every()

Verifica se todos os itens atendem à condição.

```
const notas = [7, 8, 10];

const todosAprovados = notas.every((nota) => {
  return nota >= 7;
});

// true
console.log(todosAprovados);
```

16.7 - reduce()

Acumula valores em um resultado final.

```
const numeros = [1, 2, 3, 4];

const soma = numeros.reduce((acumulador, numero) => {
  return acumulador + numero;
}, 0);

// 10
console.log(soma);
```

Esses métodos fazem parte da API de Array e estão entre os recursos mais importantes da linguagem para iteração e transformação de coleções.

17 - Function: função como valor e como referência

Função em JavaScript também é um tipo de referência. Mais do que isso: função pode ser armazenada em variável, passada como parâmetro e retornada por outra função. Isso será aprofundado na próxima aula, mas aqui já vale estabelecer a base.

Exemplo de função em variável:

```
const saudacao = () => {
  console.log("Olá");
};

// Olá
saudacao();
```

Exemplo de função recebendo função como parâmetro:

```
function executar(acao) {
  acao();
}

// Executando ação
executar(() => {
  console.log("Executando ação");
});
```

18 - Parâmetro e retorno

Parâmetro é a informação que a função recebe na definição. Retorno é o valor que ela devolve ao final da execução.

Exemplo:

```
function somar(a, b) {
  return a + b;
}
```

- a e b são parâmetros
- return a + b é o retorno

Uso:

```
const resultado = somar(2, 3);
```

```
// 5
console.log(resultado);
```

19 - Tempo de declaração e tempo de execução

Esse é um conceito didático importante para evitar confusão.

- Tempo de declaração: quando estamos definindo a função.
- Tempo de execução: quando a função é efetivamente chamada.

Exemplo:

```
function dizerOi() {
  console.log("Oi");
}
```

Aqui a função foi declarada, mas ainda não executou nada.

Só executa quando chamamos:

```
// Oi
dizerOi();
```

Esse ponto ajuda bastante a entender por que passar a função como parâmetro não é a mesma coisa que executar função na hora.

Exemplo:

```
function executar(fn) {
  fn();
}

function teste() {
  console.log("Teste");
}
```

```
// Teste
executar(teste);
```

Aqui estamos passando a referência da função teste. A execução acontece depois, dentro de executar.

Ao final dessa parte, vale apenas citar de forma breve que funções em JavaScript são first class functions, e que funções que recebem ou retornam outras funções são chamadas de high order functions. No uso prático desta aula, isso aparece muito como callbacks, tema que será aprofundado na próxima aula.

20 - Try/catch

Para finalizar a aula, vale apresentar uma primeira forma de tratamento de erro: try/catch.

A ideia é simples:

- o bloco try contém o código que pode gerar erro;
- o bloco catch captura o erro, evitando que ele interrompa o programa sem tratamento.

Exemplo:

```
try {
  const usuario = JSON.parse("{ nome: 'Ana' }");
  console.log(usuario);
} catch (erro) {
  // Ocorreu um erro ao converter JSON
  console.log("Ocorreu um erro ao converter JSON");
  console.log(erro.message);
}
```

Outro exemplo mais simples:

```
try {
  throw new Error("Erro proposital");
} catch (erro) {
```

```
  // Mensagem do erro: Erro proposital
  console.log("Mensagem do erro:", erro.message);
}
```

21 - Conclusão

Nesta aula, a ideia central foi entender que tipos de referência não guardam apenas um valor isolado, mas uma referência para uma estrutura em memória. A partir disso, fez sentido estudar o que é um objeto em JavaScript, como object, array e function entram nessa lógica, e por que comparação e cópia de referências costumam gerar confusão no começo.

Também vimos que JavaScript oferece APIs prontas para trabalhar com dados, o que ajuda a entender a ideia prática de que a linguagem possui métodos e propriedades para strings, numbers, arrays e objetos. Em especial, arrays trazem uma coleção muito importante de métodos para repetição e transformação de dados, como forEach, map, filter, find e reduce.

Por fim, encerramos com try/catch, que abre o caminho para um tema muito importante no backend: tratamento de erro. Na próxima aula, o aprofundamento natural é em funções, callbacks e o papel delas como valor, argumento e base de várias construções da linguagem e do Node.js.

22 - Referências

- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Data_structures
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Object
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Array
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Operators/Spread_syntax
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference
- https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON
- https://nodejs.org/en/learn/getting-started/introduction-to-nodejs

-
https://nodejs.org/en/learn/getting-started/how-much-javascript-do-you-need-to-know-to-use-nodejs

Questões do TA:

QUESTÃO 1

Em JavaScript, qual alternativa explica corretamente a diferença entre tipos primitivos e tipos de referência?

A) Tipos primitivos sempre ficam no heap, enquanto tipos de referência sempre ficam na stack.

B) Tipos primitivos guardam diretamente o valor, enquanto tipos de referência guardam uma referência para um valor em memória.

C) Tipos primitivos só existem no navegador, e tipos de referência só existem no Node.js.

D) Tipos de referência não podem ser comparados entre si.

E) Tipos primitivos podem ter métodos, mas tipos de referência não.

Gabarito: B)

Misturar as alternativas? (x) Sim ( ) Não

QUESTÃO 2

Sobre objetos em JavaScript, assinale a alternativa correta.

A) Um objeto em JavaScript só pode armazenar números e strings.

B) Um objeto em JavaScript é uma estrutura que agrupa apenas funções.

C) Um objeto em JavaScript pode guardar propriedades e métodos.

D) Um objeto em JavaScript é igual a JSON.

E) Um objeto em JavaScript não pode ser alterado depois de criado.

Gabarito: C)

Misturar as alternativas? (x) Sim ( ) Não

QUESTÃO 3

Um aluno criou dois objetos separados com os mesmos dados e quis verificar se eles eram “iguais” usando o operador de comparação. Considerando que, em JavaScript, tipos de referência são comparados pela referência de memória, assinale a alternativa em que o resultado da comparação é true.

A)

```
const aluno1 = { nome: "Ana" };
const aluno2 = { nome: "Ana" };
aluno1 === aluno2
```

B)

```
const lista1 = [1, 2, 3];
const lista2 = [1, 2, 3];
lista1 === lista2
```

C)

```
const usuario1 = { id: 1 };
const usuario2 = { id: 1 };
usuario1 == usuario2
```

D)

```
const pessoa = { nome: "Carlos" };
const copia = pessoa;
pessoa === copia
```

E)

```
const nome1 = new String("Ana");
const nome2 = new String("Ana");
```

```
nome1 === nome2
```

Gabarito: D)

Misturar as alternativas? (x) Sim ( ) Não

QUESTÃO 4

Sobre JSON, API e métodos nativos, assinale a alternativa correta.

A) JSON é uma função usada para repetir blocos de código.

B) API significa Array Programming Interface.

C) JSON.parse() transforma um valor JavaScript em texto JSON.

D) JSON.stringify() transforma texto JSON em array.

E) API é uma interface pronta de recursos que podemos usar, e JSON é um formato de texto para representar dados.

Gabarito: E)

Misturar as alternativas? (x) Sim ( ) Não